Guia técnico — Parte do Aluno 2

Programação Distribuída | identidade, overlay e C4 | revisão de

20/10/2026

GUIA EXPLICATIVO

Fluxos, módulos, funções, decisões, evidências e pendências

Leitura baseada no código e na documentação do repositório

Revisão: 20/10/2026

Uso didático: implementação atual separada de requisitos e trabalho futuro

1. Objetivo e leitura do estado

Este guia explica a identidade dos nós, as APIs de membro, o overlay Chord e a implementação C4 de eleição/log replicado. Ele distingue as APIs locais do tráfego TCP e apresenta, separadamente, requisito previsto, comportamento implementado, evidência de execução e pendência.

A divisão evoluiu ao longo dos checkpoints. O Aluno 2 mantém a base de identidade e cadastro do Super Peer, a API local de metadados/índice e o overlay Chord. O serviço C3 de membership, heartbeat, confirmação e Gossip é responsabilidade integrada do Aluno 1; o Aluno 2 fornece as operações Chord que esse serviço invalida/repara. No C4, c4_cluster.c liga Bully e replicação do ANNOUNCE ao Super Peer.

2. Arquitetura e fronteiras

```
node.c: configuração + UUID --SHA-256--> NodeID de 32 bytes
                                            |
                                            v
Super Peer: membership + metadados locais + Chord + C4
     |                          |                        |
     +- JOIN/LEAVE C1/C2        +- Chord 20–24         +- C4 15–19
     |                          |                        |
     +- tabela local            +- lookup de chave      +- log/eleição

Peer ativo -- JOIN/ANNOUNCE/LOOKUP --> Super Peer
Super Peers -- heartbeat/Gossip -----> membership e reparo Chord
```

bin/superpeer e bin/peer são processos diferentes. bin/node e bin/client são aliases para compatibilidade. O Super Peer recebe os protocolos pelo mesmo listener TCP e encaminha cada tipo ao módulo correspondente. A API local e a integração pela rede são camadas diferentes: chamar uma função C não demonstra que o framing, o socket, a identidade remota e a resposta funcionam.

3. Identidade: configuração, UUID e NodeID

Como a identidade é calculada

```
IP normalizado em bytes + porta em ordem de rede + UUID de 16 bytes
                              | SHA-256
                              v
                        NodeID de 32 bytes
```

node_config_validate valida porta e endereço; node_config_init_with_uuid normaliza e copia uma identidade conhecida; node_generate_uuid obtém 16 bytes aleatórios; node_compute_id calcula SHA-256; node_init monta o Node com PID e papel; node_validate recalcula a identidade e confere coerência. node_id_compare dá a ordem lexicográfica usada como prioridade Bully. Conversores hexadecimais são usados em diagnóstico e CLI.

O processo persiste node.uuid por porta para manter o NodeID depois de reiniciar. O PID identifica apenas o processo local: não é identidade remota nem é enviado. NodeID coerente valida a relação entre dados e hash, mas não autentica criptograficamente uma máquina.

Decisões

- UUID estável, IP/porta em ordem de rede e hash binário evitam que o identificador dependa da grafia do endereço ou da ordem de bytes do host.
- O wire transporta campos definidos e não a memória bruta de struct Node.
- O papel Peer/Super Peer não faz parte do NodeID, para que a identidade dependa do endpoint e da instância, não de um campo de aplicação.

4. Cadastro de membros: API local e JOIN TCP

membership.c encapsula SuperPeer e preserva funções públicas declaradas em superpeer.h. superpeer_create registra o próprio Super Peer; superpeer_register_node valida NodeID e inclui ou atualiza; superpeer_find_member devolve uma cópia; superpeer_member_count conta membros ativos segundo a regra vigente; superpeer_unregister_node registra a saída/remoção apropriada. O mutex protege busca e mutação. membership_snapshot devolve uma cópia que o chamador libera.

Esse bloco constitui a API local. Na integração via rede, superpeer.c recebe JOIN, valida origem/destino e o payload explícito, reconstrói NodeConfig, recalcula NodeID e só retorna ACK depois do cadastro. directory.c adapta ANNOUNCE/LOOKUP ao índice e à tabela de membros. O formato JOIN legado usa descritor de 64 bytes; as extensões C3 acrescentam incarnação sem alterar a resposta ACK antiga.

Não confundir:

- superpeer_register_node: chamada local de cadastro;
- handler TCP JOIN: recebe e valida mensagem wire e então chama a API;
- membership_merge: aplica uma observação de Gossip segundo incarnação, sequência, estado e origem;
- chord_network_join: ingresso no anel Chord, não é JOIN de Peer.

5. Metadados e índice C2

metadata.c mantém uma API local para documentos e localizações; directory.c transforma o protocolo de rede para essa API. A FileMetadata e o wire não devem ser tratados como a mesma estrutura. Um documento pode ter registros de localização por chunk; a consulta local resolve os NodeIDs com a tabela de membros e filtra disponibilidade conforme o estado.

Na versão integrada, ANNOUNCE chega depois que o Peer finaliza o upload. O Super Peer registra metadados e NodeID proprietário, mas não armazena o PDF. O download usa as localizações devolvidas e conecta diretamente aos Peers. Uma API local de metadata não prova lookup distribuído: Chord encontra o responsável por uma chave, mas a consulta de documentos continua local e não migra índices.

Funções para seguir: metadata_create, metadata_announce, metadata_find_document, metadata_remove_peer; em directory.c, os handlers de ANNOUNCE/LOOKUP, e em superpeer.c, process_announcement e answer_lookup.

6. Checkpoint 3: Chord e integração com falhas

Modelo de overlay

Cada Super Peer tem NodeID de 256 bits e ocupa uma posição circular. A API local mantém sucessor, predecessor e 256 fingers. chord_id_add_power calcula (id + 2^i) mod 2^256; chord_interval_open_closed e chord_interval_open_open comparam intervalos circulares; chord_route escolhe o próximo salto a partir da tabela local.

Funções da API local: chord_create, chord_local, chord_get_successor, chord_get_predecessor, chord_get_finger, chord_set_successor, chord_notify, chord_consider_predecessor, chord_set_finger e chord_route. chord_forget invalida referências e bloqueia NodeID indisponível; chord_allow libera depois de confirmação de instância atual; chord_repair escolhe candidato vivo em ordem circular.

Integração TCP

chord_network.c implementa o wire e o tráfego real. chord_network_join contata bootstrap; chord_network_find faz lookup de uma chave pelo overlay; chord_network_maintain executa tarefas de manutenção/finger. chord_network_handle valida as mensagens recebidas e responde pelo listener Super Peer. Os tipos Chord são 20–24, no mesmo header C1/C2 de 98 bytes.

O primeiro registro do inventário pode ser bootstrap. Outros Super Peers tentam entrar no anel; o Super Peer mantém stabilize/notify/fix_fingers. O limite de saltos, conjunto visitado, prazo de RPC e alternativas evitam loops e chamadas indefinidas.

Relação com o membership do Aluno 1

O detector C3 do Aluno 1 julga ALIVE/SUSPECT/FAILED/REMOVED com heartbeat, sondagens e Gossip. Ao confirmar FAILED/REMOVED, o integrador chama chord_forget; isso limpa sucessor/predecessor/fingers. Uma RPC iniciada antes da falha pode terminar atrasada, então o NodeID bloqueado não deve aceitar resposta velha. chord_allow ocorre após evidência direta de ALIVE da instância atual. chord_repair refaz sucessor circular; a manutenção Chord recompõe as demais referências.

Isso repara topologia, não o índice de documentos. Gossip é disseminação de membership, não consenso; JOIN de Peer não equivale a ingresso Chord.

7. Checkpoint 4: Bully, operation log e replicação

Eleição esperada

- Um membro inicia eleição quando o cluster começa ou quando o líder observado fica FAILED.
- Envia M_ELECTION aos Super Peers ativos com NodeID maior, incluindo termo e posição conhecida do log.
- Um candidato superior responde M_OK e inicia sua própria eleição.
- Se não houver resposta de candidato superior, o nó assume liderança e anuncia M_COORDINATOR.
- Os membros aceitam o coordenador se termo e prioridade forem válidos.

No código, c4_cluster_tick coordena as tentativas, c4_cluster_handle recebe eleição/coordenador/recuperação, c4_become_coordinator recupera entradas e publica liderança, e c4_cluster_is_leader/c4_cluster_get_leader permitem consultar o estado. Desde 20/10/2026, cada rodada registra C4 ELECTION no log para deixar visível a evidência operacional.

Log persistente

c4_log.c guarda registros append-only C4LG. Cabeçalho: índice u64, versão u64, TransactionID de 16 bytes, tamanho u32 e CRC32; depois vem o payload. c4_log_open valida e carrega; c4_log_append cria entrada; c4_log_append_entry replica uma entrada já formada; encode/decode serializam wire; c4_log_get lê por índice; c4_log_get_transaction suporta idempotência; getters expõem último índice/versão.

Uma repetição do mesmo TransactionID e mesmo conteúdo não duplica entrada; conteúdo diferente com o mesmo ID gera conflito. Ao iniciar, cada Super Peer revalida o arquivo e reaplica suas entradas ao diretório.

ANNOUNCE replicado

O fluxo C4 ligado ao produto é ANNOUNCE C2. process_announcement exige Peer cadastrado. Se o Super Peer não é líder, encaminha a operação ao coordenador. O líder grava a operação, aplica localmente e envia registro via M_STATE_TRANSFER às réplicas. Um candidato eleito consulta registros com M_SNAPSHOT, recupera os que faltam, anuncia liderança e tenta sincronizar logs.

Tipos C4: M_ELECTION (15, 24 bytes), M_OK (16, sem corpo), M_COORDINATOR (17, 16 bytes), M_SNAPSHOT (18, consulta por índice) e M_STATE_TRANSFER (19, registro C4LG). Todos usam framing, TransactionID e CRC existentes; o registro persistente também tem checksum próprio.

8. Escolhas, garantias e limites

- NodeID lexicográfico dá prioridade total e determinística a Bully; não fornece autenticação nem quórum.
- Chord mantém topologia em estruturas locais sob mutex e faz RPC fora do mutex; reduz contenção e evita esperar por rede segurando trava.
- Prazos de RPC limitam espera por conexões, envio e recebimento lento.
- Log com índice, versão e TransactionID permite replay, recuperação e deduplicação de operações.
- Snapshot transfere operações ausentes por índice; state transfer mantém réplica do log e aplica ANNOUNCE no nó destino.
- O líder aplica localmente antes de todas as confirmações. Interrupção durante replicação pode deixar estado parcial.
- Termo C4 é mantido em memória, não persistido.
- A implementação de Bully mais log replicado não prova consenso forte. Falhas por parada não cobrem partições; não se comprovam disponibilidade de 99,5%, consistência forte ou metas globais de desempenho.
- C5 ainda deve trazer 2PC e IST; ambos permanecem pendentes.

9. Evidências e limites dos testes

Os registros do projeto documentam API local de identidade/membership, testes locais C2/C3, Chord TCP com três Super Peers, reparo com cinco processos e integração C4 via runc4.sh com teste local do log e 11 verificações TCP. Esses resultados têm escopos diferentes: API local prova comportamento de função; integração TCP prova os fluxos exercitados entre processos; nenhum desses resultados prova tolerância a partições ou todas as execuções possíveis.

Em 20/10/2026, make all recompilou o binário e bash tests/c4/run.sh passou 4/4 quando executado com permissão para sockets locais. O código passou a emitir C4 ELECTION, que era o texto esperado pelo roteiro. Esse runner chama bin/client --cmd put e --cmd state, opções rejeitadas pelo cliente atual e ignoradas pelo roteiro; portanto esse PASS não comprova PUT replicado. A validação C4 específica documentada para runc4.sh é evidência separada. O roteiro não foi alterado.

10. Demonstração e explicação curta

Para demonstrar o overlay com três Super Peers, em terminais separados:

```
make
./bin/superpeer --port 55101
./bin/superpeer --port 55111 --chord-host 127.0.0.1 --chord-port 55101
./bin/superpeer --port 55121 --chord-host 127.0.0.1 --chord-port 55101
```

Use inventário de configuração se quiser reproduzir a seleção de nó por porta. O comando topology mostra referências do overlay; lookup --object-id roteia uma chave Chord e não consulta automaticamente todos os catálogos.

“O NodeID transforma IP, porta e UUID em uma identidade estável. O Super Peer valida JOIN e mantém os membros. No C3, Chord organiza Super Peers numa faixa circular para rotear chaves; o detector de falhas pode invalidar vizinhos e o Chord repara a topologia. No C4, o maior NodeID ativo tem prioridade Bully. O líder registra ANNOUNCEs num log persistente e tenta replicá-los; nós eleitos recuperam entradas por índice. Como não há commit atômico nem termo persistido, isso melhora recuperação, mas não equivale a consenso forte.”

11. Referências internas

Fontes principais: requisitos_aluno_2.md, explicacao_codigo_aluno_2.md, checkpoint_4_implementacao.md, continuidade_checkpoint_3_aluno_1.md, integracao_checkpoint_3.md, headers e implementações de node, membership, chord, chord_network, c4_cluster, c4_log, directory e superpeer.